BASIC I/O System Design

I/O device → peripherals

device → up : input (IN)

up → device : output (OUT)

Addr. bus : 20 bits (A0-15 + A16-19)

But

- I/O device/port को locate करने के लिए 8086 uses 16 bits (AD0-15)
  ∴ I/O ports = 2^{16} = 65,536 → 0000h - FFFFh
- I/O से संबंधित registers = AL / AX

I/O Controller → Interface bet^n sys. bus & I/O device

- I/O instructions

I/O device identified using 2 METHODS

① Fixed address (Direct I/O address)
- port address fixed inside the instruction itself
- uses 8 bit port addr. (can access from 00-FFh)
  - 256 ports
  - 1st (00) to 256th (FF) port

IN AL, 60H
OUT 20H, AL

IN AX, 06H
up reads 16 bit word from I/O port address 0006H

② Variable Addressing (Indirect I/O address)
- port addr. stored in DX
- uses 16 bit port addr.
  - 0100 - FFFFh
  - 257th to 65,536th port

MOV DX, 03F8h
IN AL, DX

AX = D_{15-8} : D_{7-0}
AH AL

Accessing I/O Devices

① Memory-mapped I/O

- I/O device assigned an addr. from the memory addr. space
- up treats an I/O device like a memory loc.

→ 8086 has 2^{20} = 1MB. W mem-mapped I/O, some addresses from this 1MB are reserved for I/O devices.

Suppose, an I/O device is assigned PA = A0000H

up can communicate w it using normal mem. ins.:

```
MOV AL, [A0000H] ; read
MOV [A0000H], AL ; write
```

The device is accessed as if it were RAM/ROM

Disadv: Every addr. assigned to an I/O device reduces the available mem. space

Say, if a device occupies 100 address, (1MB - 100) addresses remain available for actual mem.

Adv: Since I/O device looks like mem, so memory instructions can be used. NOT ltd to IN and OUT.

② Isolated I/O (port-mapped I/O)

I/O mapped I/O or

- I/O devices have a separate addr. space from memory
- Device is not assigned a mem addr. It is assigned an I/O port addr.

Adv! Does not reduce memory space

00000H — mem space — FFFFFH
0000H — I/O space — FFFFH
} separate

mem sp: 0000H
I/O sp: 0000H
} poss.

How does HW know which device to select?

Suppose, address = 0005H is present on addr. bus
How does up know if it is mem. loc. = 00005H or I/O port = 0005H?

Ans: Control signal $M/\overline{IO}$

Address = 0005H
Mem. operat^n: $M/\overline{IO} = 1$ → Mem decoder becomes active

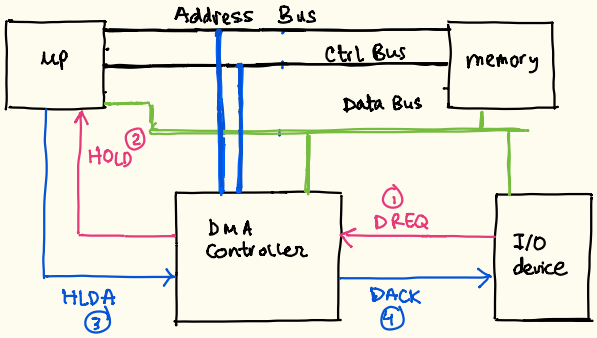
I/O operat^n: Address = 0005H
$M/\overline{IO} = 0$ → I/O decoder becomes active

I/O Data Transfer Technique

1. Programmed I/O
2. Interrupt-driven I/O
3. Direct Memory Access (DMA)

Direct Memory Access (DMA)

external device that takes over control of the system bus from the µp & facilitates data transfer between memory and I/O devices.



1. Peripheral req. DMA Controller to provide access for the system bus.
2. DMA Controller sends HOLD req to µp asking access for system bus
3. µp sends Hold Acknowledgement (HLDA) to DMAC providing access to system bus
4. DMAC sends DMA Acknowledgement (DACK) to peripherals after getting access to the system bus

8237 DMAC → 4 channel device

DREQ 0 ··· 3

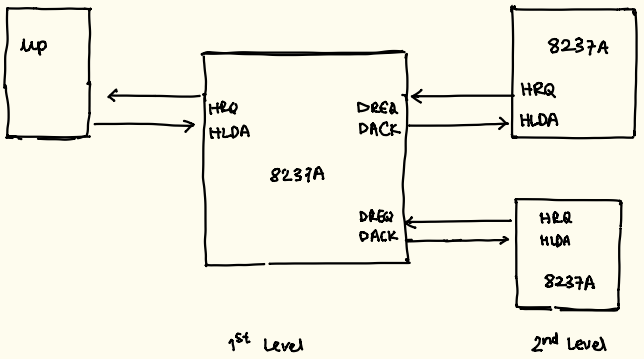
DACK 0 ··· 3

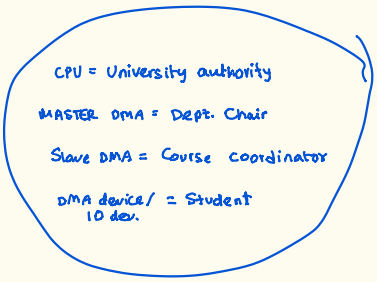
8237 DMAC → 4 channel device

DREQ 0 ··· 3

DACK 0 ··· 3

DMA Cascade Mode





master DMA → bus arbitration + slave selection

slave DMA → performs the actual DMA transfer

① Peripheral requests the slave

Device → DREQ of slave

The slave determines which of its DMA channel has requested service

② Slave requests the master by asserting its HRQ signal

Slave HRQ → Master DREQ

③ Master requests control of the system bus

Master HRQ → CPU HOLD

The CPU completes its current bus operation and responds : CPU HLDA → Master

Now DMAC has control of the system bus

④ Master selects the requesting slave & activates corresponding DACK line :

Master DACK → Slave

⑤ Slave performs the actual DMA transfer.

It generates the necessary : memory addr., R/W ctrl signals, transfer handling